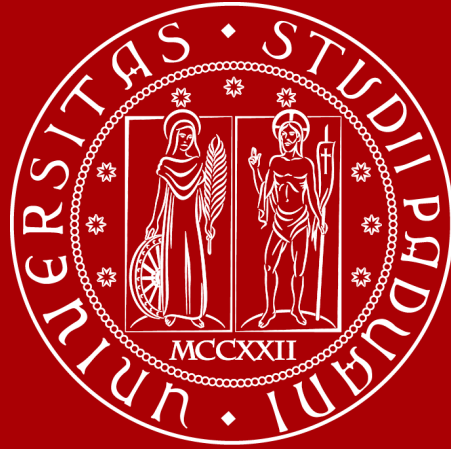




UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Distributed Algorithms: Mini-batch k-Means

Kevin Brugnera, Riccardo Ferrante, Marco Lorenzato, Federico Scianna



- **Introduction (*Federico*)**
 - Mini-batch k-Means
 - Distributed Processing Framework
- **Data Processing (*Kevin*)**
 - The RCV1 Dataset
 - Spark Execution Workflow
- **Results and Benchmarks**
 - **Part I (*Riccardo*)**
 - Cluster Topology, Data Partitioning, Strong Scaling
 - **Part II (*Marco*)**
 - Weak Scaling, k-Means Comparison, Cluster Evaluation



Spark implementation of MB k-Means

Algorithm 1 Mini-batch k -Means.

```
1: Given:  $k$ , mini-batch size  $b$ , iterations  $t$ , data set  $X$ 
2: Initialize each  $\mathbf{c} \in C$  with an  $\mathbf{x}$  picked randomly from  $X$ 
3:  $\mathbf{v} \leftarrow 0$ 
4: for  $i = 1$  to  $t$  do
5:    $M \leftarrow b$  examples picked randomly from  $X$ 
6:   for  $\mathbf{x} \in M$  do
7:      $\mathbf{d}[\mathbf{x}] \leftarrow f(C, \mathbf{x})$  // Cache the center nearest to  $\mathbf{x}$ 
8:   end for
9:   for  $\mathbf{x} \in M$  do
10:     $\mathbf{c} \leftarrow \mathbf{d}[\mathbf{x}]$  // Get cached center for this  $\mathbf{x}$ 
11:     $\mathbf{v}[\mathbf{c}] \leftarrow \mathbf{v}[\mathbf{c}] + 1$  // Update per-center counts
12:     $\eta \leftarrow \frac{1}{\mathbf{v}[\mathbf{c}]}$  // Get per-center learning rate
13:     $\mathbf{c} \leftarrow (1 - \eta)\mathbf{c} + \eta\mathbf{x}$  // Take gradient step
14:   end for
15: end for
```

```
def minibatch_kmeans(rdd_train, k: int, b: int, epochs: int, seed: int):
    """Executes the Mini-Batch K-Means algorithm using Spark."""
    centers_list = rdd_train.takeSample(False, k, seed)
    centers = np.array(centers_list)

    v = np.zeros(k)

    # Calculate fraction for Spark's .sample()
    total_count = rdd_train.count()
    fraction = min(1, float(b)/total_count)

    for epoch in range(epochs):
        # Broadcast the centers
        bc_centers = rdd_train.context.broadcast(centers)

        # Distributed sampling on workers
        rdd_batch = rdd_train.sample(False, fraction)

        # Distributed mapping --> (cluster_idx: (point,1))
        mapped_batch = rdd_batch.map(
            lambda x: (closest_idx(x, bc_centers.value), (x, 1))
        )

        # Distributed reduction, collecting only the k aggregates
        reduced_batch = mapped_batch.reduceByKey(
            lambda a, b: (a[0] + b[0], a[1] + b[1])
        ).collect()

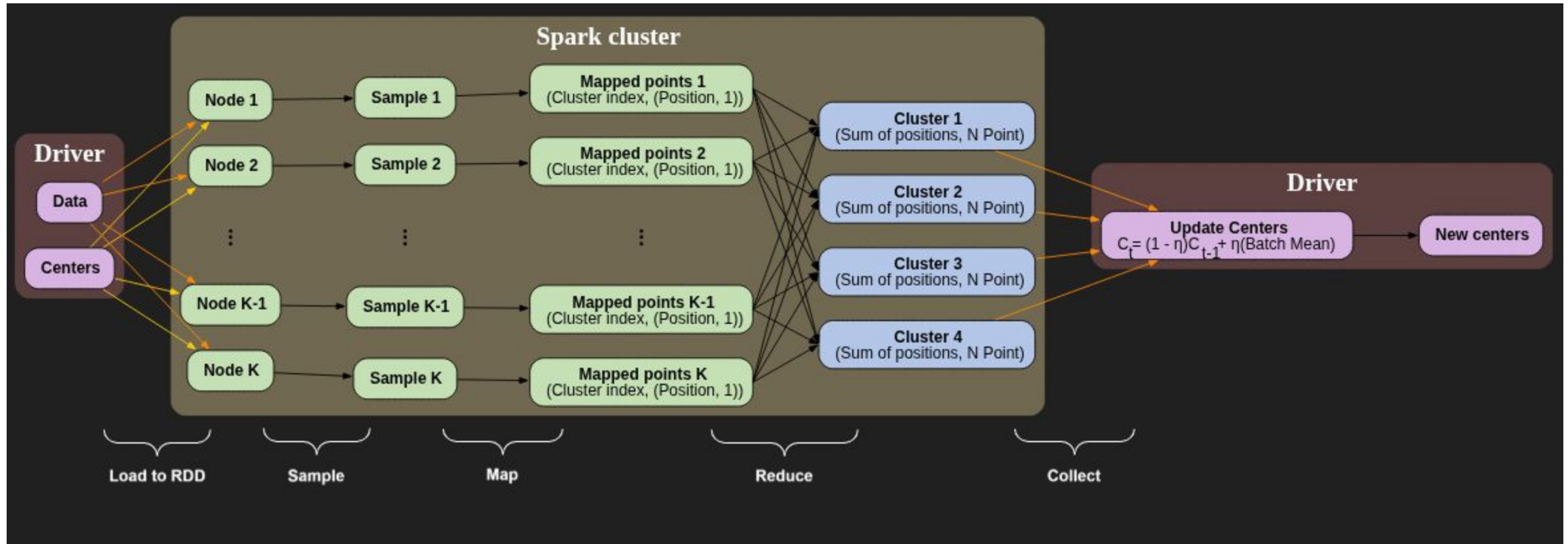
        # Local update on the Driver
        for c_idx, (sum_x, count) in reduced_batch:
            v[c_idx] += count
            eta = count / v[c_idx]
            batch_mean = sum_x / count
            centers[c_idx] = (centers[c_idx] * (1.0 - eta)) + (batch_mean * eta)

        bc_centers.destroy()

    return centers.tolist()
```

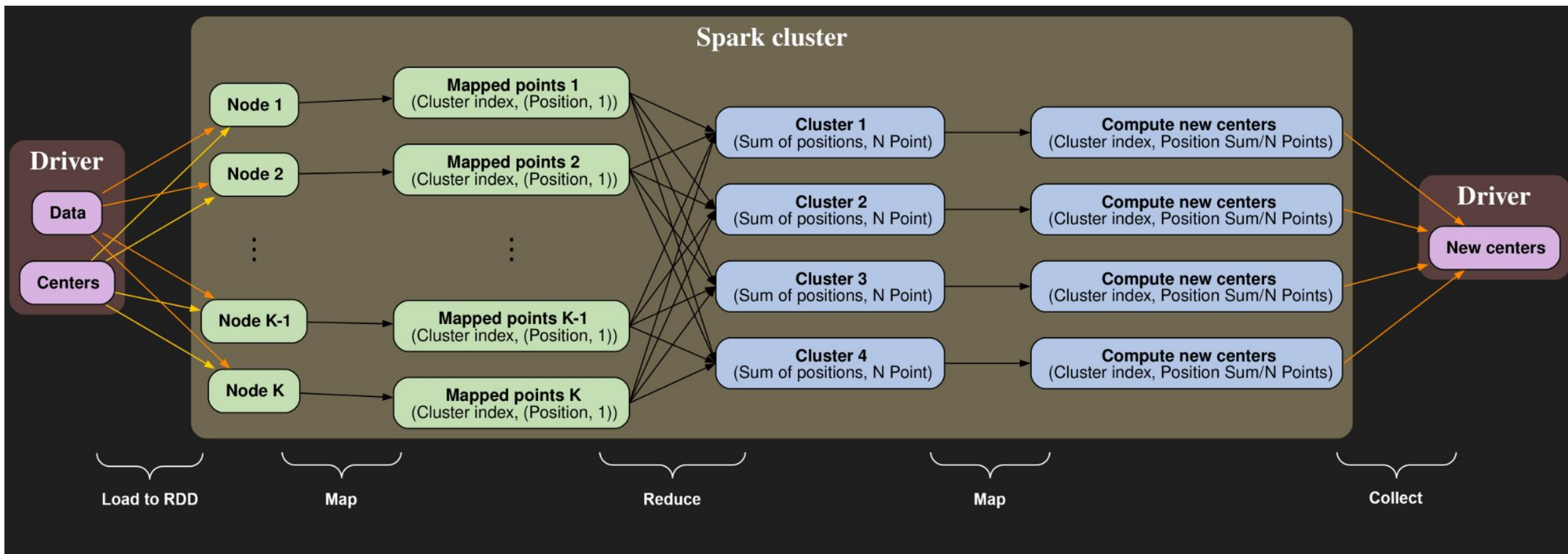


Spark implementation of MB k-Means





Spark implementation of “classic” k-Means





Tuning of the b parameter

- `b_search.py`

- Calinski-Harabasz score: $CH = \frac{BCSS/(k-1)}{WCSS/(n-k)}$

$$BCSS = \sum_{i=1}^k n_i ||\mathbf{c}_i - \mathbf{c}||^2$$

$$WCSS = \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} ||\mathbf{x} - \mathbf{c}_i||^2$$



The RCV1 Dataset

Reuters Corpus Volume I (RCV1) is an open-source dataset (by Reuters Ltd.), containing **804,414 manually categorized newswire articles**.

Retrieved using the Scikit-learn built-in `fetch_rcv1()` function, as a dictionary-like (*Bunch*) object with the following structure:

```
root
|-- data: sparse matrix (CSR)
|   |-- shape: (804414, 47236)
|   |-- representation: TF-IDF features
|
|-- target: sparse matrix (CSR)
|   |-- shape: (804414, 103)
|   |-- representation: binary topic labels (ground truth)
|
|-- target_names: ndarray
|   |-- shape: (103,)
|   |-- content: Reuters topic codes
|
|-- sample_id: ndarray
|   |-- shape: (804414,)
|   |-- content: document identifiers
|
|-- description: string
|   |-- dataset metadata
```

Features: correspond to words, associated with a semantic concept. Their values indicate their relevance within a document.

Topic Codes: identifiers of 103 semantic categories stored in `target_names`.
Binary labels:

- **1** = document belongs to the topic
- **0** = otherwise.

Macro Topics: CCAT · ECAT · GCAT · MCAT

GROUND TRUTH ASSUMPTION:

k = 4 clusters



Dimensionality Reduction

Documents are sparse vectors in a **47k+ dim. space**: most features provide **limited information** while increasing **computational cost**.

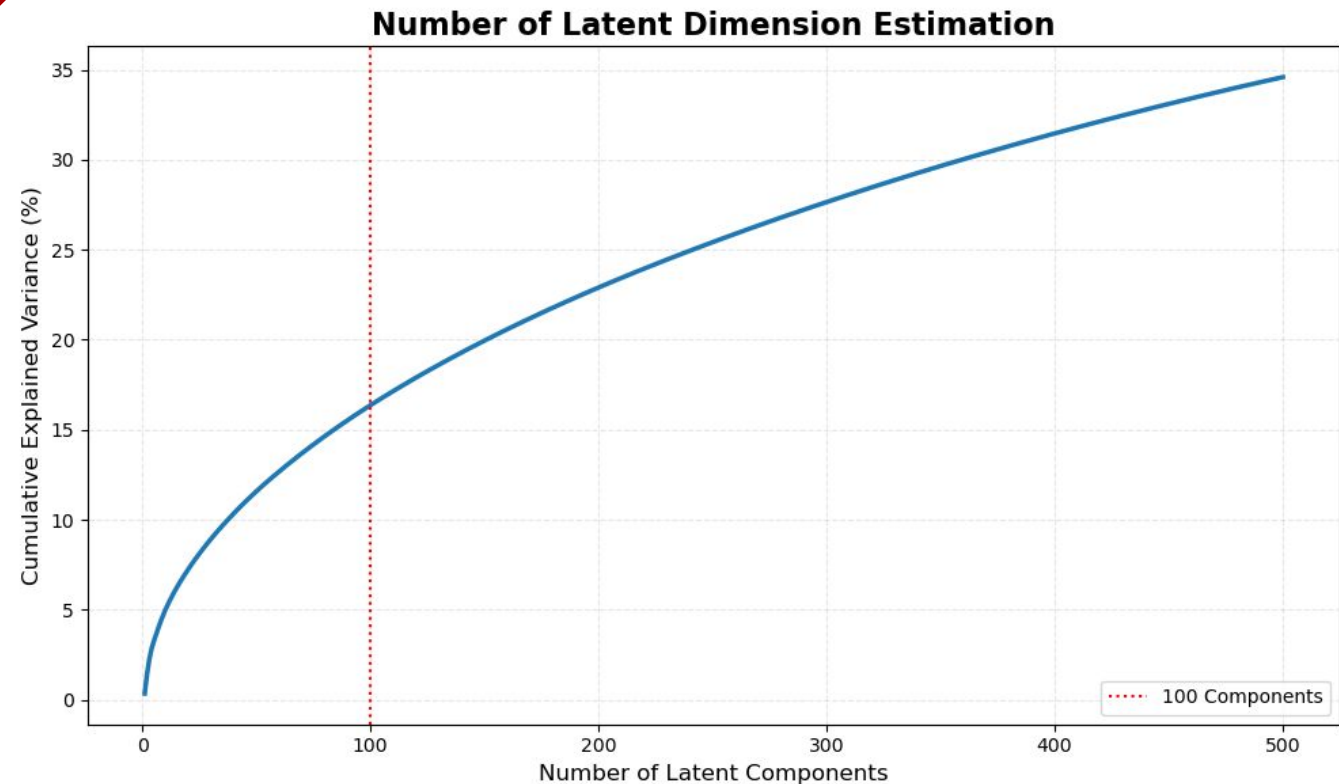
Truncated Single Value Decomposition:

applied to project documents into a lower-dimensional latent space

```
sample_idx = np.random.choice(  
    rcv1.data.shape[0],  
    size=50000,  
    replace=False  
)  
sorted_idx = np.sort(sample_idx)  
X_sample = rcv1.data[sorted_idx]  
  
# Reduce to 100 latent dimensions  
svd = TruncatedSVD(  
    n_components=100,  
    random_state=16)  
svd.fit(X_sample)
```

From `generate_data()` function
(`data_generation.py`)

Selected **100-dimensional latent space** after identifying the elbow in cumulative explained variance plot (500 SVD components)





Data Preparation

```
n_docs = rcv1.data.shape[0]
batch_size = 10000

for i, start_idx in enumerate(range(0, n_docs, batch_size)):
    end_idx = min(start_idx + batch_size, n_docs)
    print(f" -> Processing and saving chunk {start_idx} to {end_idx} / {n_docs}")

    # Transformation and normalization of current chunk
    X_batch = rcv1.data[start_idx:end_idx]
    X_dense_batch = svd.transform(X_batch)

    # This normalization avoids different cluster assignment for documents with same topics but
    X_normalized_batch = normalize(X_dense_batch, norm='l2', axis=1)

    # Saves the corresponding macro labels for the current chunk
    Y_batch = macro_labels_list[start_idx:end_idx]

    # Temporary DataFrame to store data
    df_chunk = pd.DataFrame({
        'features': list(X_normalized_batch),
        'true_labels': Y_batch
    })

    # Saves data chunk in dedicated directory in .parquet format
    chunk_filename = f"data_{i:02d}.parquet"
    df_chunk.to_parquet(os.path.join(data_dir, chunk_filename), engine="pyarrow", index=False)

    # Evaluation data saving
    eval_filename = f"eval_{i:02d}.parquet"

    # Takes only single label documents
    mask_single = df_chunk['true_labels'].apply(lambda x: len(x) == 1)
    df_eval_chunk = df_chunk[mask_single].copy()
```

The following framework was used to generate and store the RCV1 **Training Dataset**.

Given a 10k-sample batch:

- Apply **TruncatedSVD** → latent representation
- Extract corresponding **macro-labels**
- Build a structured **DataFrame**
- Store as **data_##.parquet**
- Saved in **/data** folder

Evaluation Dataset

Documents with a unique macro-label were filtered and stored with the same structure as **eval_##.parquet**, used for clustering evaluation.

From `generate_data()` function
(`data_generation.py`)



Cluster Configuration

☐	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
☐	MAPDB_2612_worker4	ubuntu-22.04-jammy-server-2026-06-17	10.67.22.226	cloudveneto.large	KRMF_key	Active	nova	None	Running	6 days, 1 hour	Create Snapshot ▼
☐	MAPDB_2612_worker3	ubuntu-22.04-jammy-server-2026-06-17	10.67.22.23	cloudveneto.large	KRMF_key	Active	nova	None	Running	1 week, 4 days	Create Snapshot ▼
☐	MAPDB_2612_worker2	ubuntu-22.04-jammy-server-2026-06-17	10.67.22.84	cloudveneto.large	KRMF_key	Active	nova	None	Running	1 week, 4 days	Create Snapshot ▼
☐	MAPDB_2612_worker1	ubuntu-22.04-jammy-server-2026-06-17	10.67.22.129	cloudveneto.large	KRMF_key	Active	nova	None	Running	1 week, 4 days	Create Snapshot ▼
☐	MAPDB_2612_master	ubuntu-22.04-jammy-server-2026-06-17	10.67.22.135	cloudveneto.medium	KRMF_key	Active	nova	None	Running	1 week, 4 days	Create Snapshot ▼

HEAD NODE (Master)

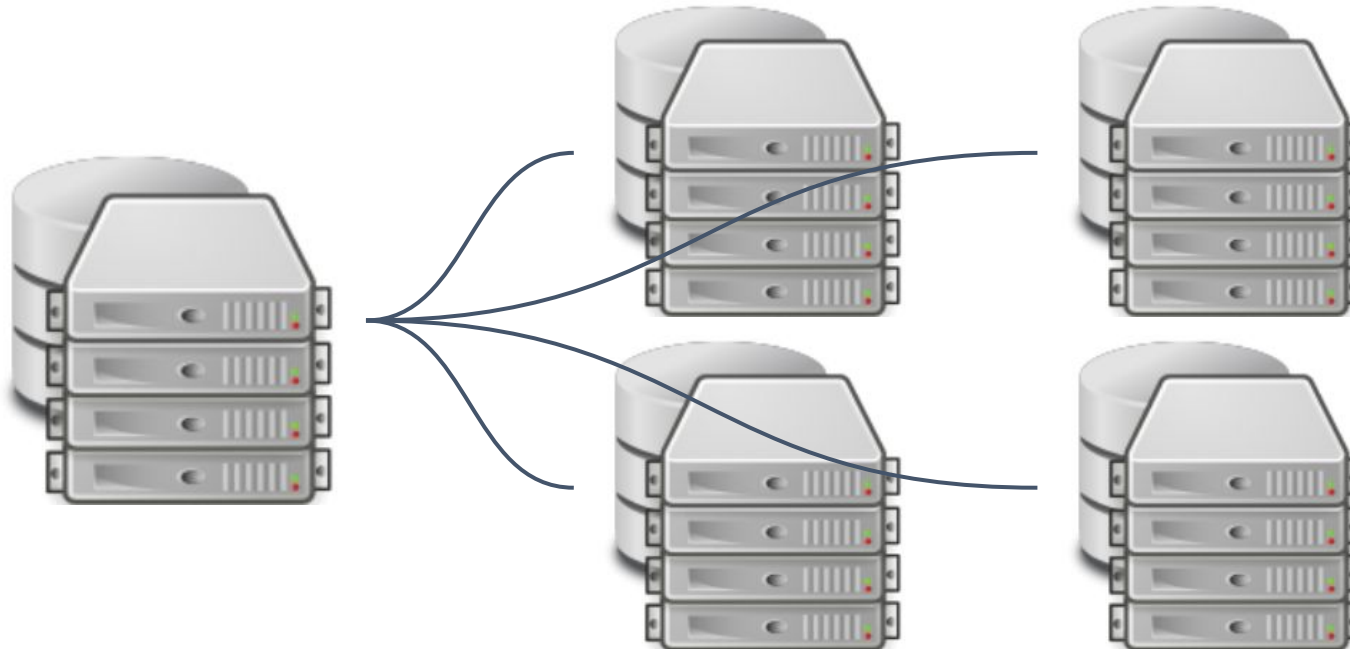
IP address:

10.67.22.135

CPUs: 2 cores

Memory: 8 GB RAM

Disk: 25 GB



WORKER NODEs

IP addresses:

10.67.22.129,

10.67.22.84,

10.67.22.23,

10.67.22.226

CPU: 4 cores

Memory: 8 GB RAM

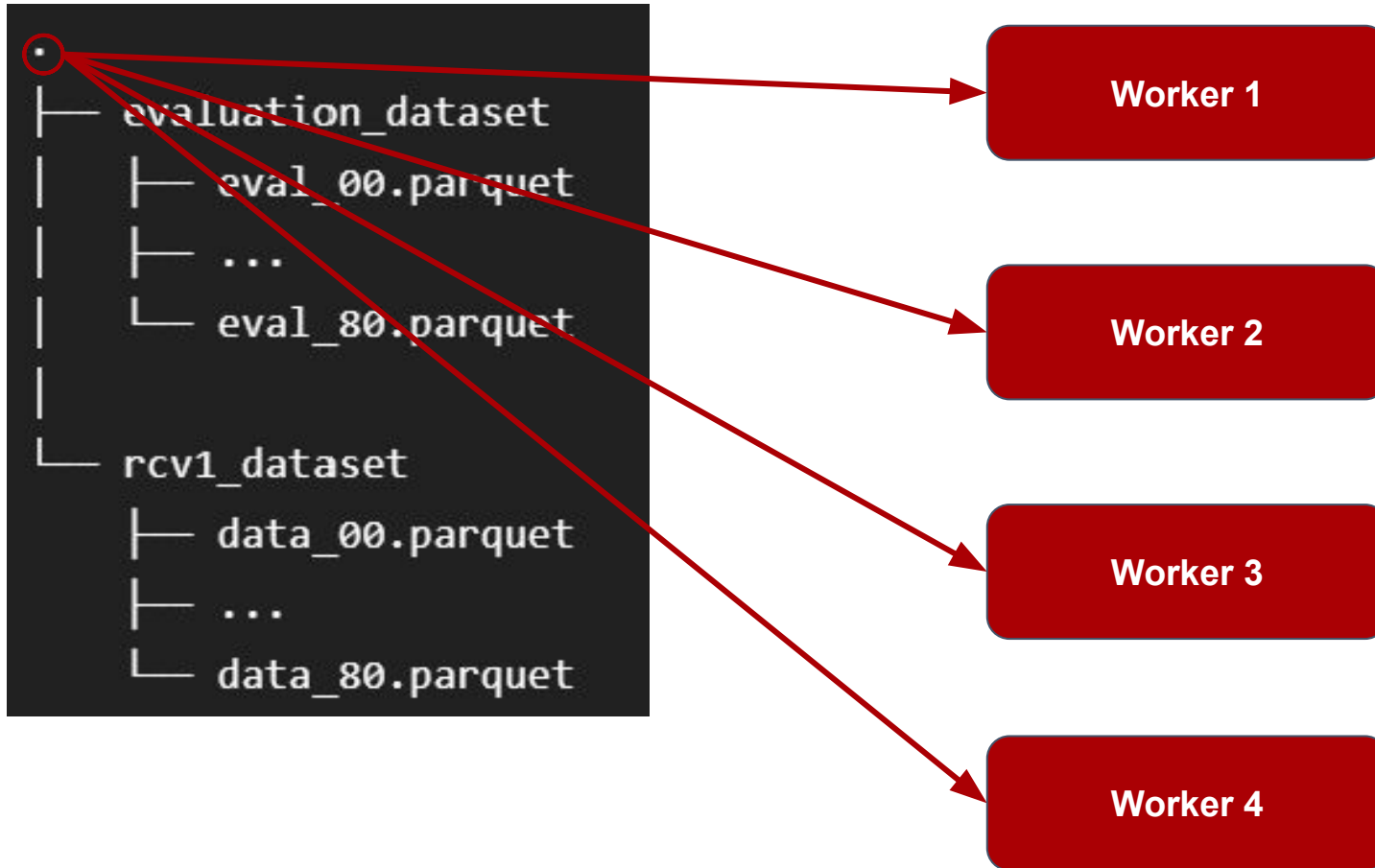
Disk: 25 GB

(per node)



Data Distribution

Running `data_generation.py` on the Master node generates the `/data` folder with the following structure:



Replicated across all processing nodes with:

```
rsync_command = [  
    "rsync",  
    "-az",  
    "--delete",  
    "-e",  
    "ssh -o StrictHostKeyChecking=no -o BatchMode=yes",  
    f"{abs_path}/data/",  
    f"{worker_user}@{ip}:{abs_path}/data/"  
]
```

From `generate_data()` function
(`data_generation.py`)



Spark Execution Workflow

The parallel-processing workflow is entirely **script-based**: after configuring the environment (`setup.sh`) and generating the dataset (`data_generation.py`), the system is **ready for distributed computation**.

- Configure Spark Application resources in `run_script.sh` (*executors, memory, cores, ...*)
- Initialize the Spark Session inside `target_script.py` (*batch_initialization.py, benchmark.py ..*)

From `run_script.sh`

```
spark-submit \  
  --master spark://$MASTER_IP:7077 \  
  --driver-memory 2G \  
  --executor-memory 5G \  
  --conf spark.executor.memoryOverhead=2048M \  
  --conf spark.memory.fraction=0.8 \  
  --conf spark.memory.storageFraction=0.7 \  
  --total-executor-cores 16 \  
  --executor-cores 4 \  
  --conf spark.eventLog.enabled=true \  
  --conf spark.eventLog.dir=file:///tmp/spark-events \  
  --py-files functions.py \  
  $TARGET_SCRIPT
```

From `Target_script.py`

```
#Creates spark session with run_script.sh specs  
spark = SparkSession.builder \  
  .appName("Target_Script") \  
  .config("spark.api.mode", " classic") \  
  .getOrCreate()
```

```
ubuntu@mapdb-2612-master:~/Project$ \  
> ./run_script.sh Target_script.py \  
^C
```

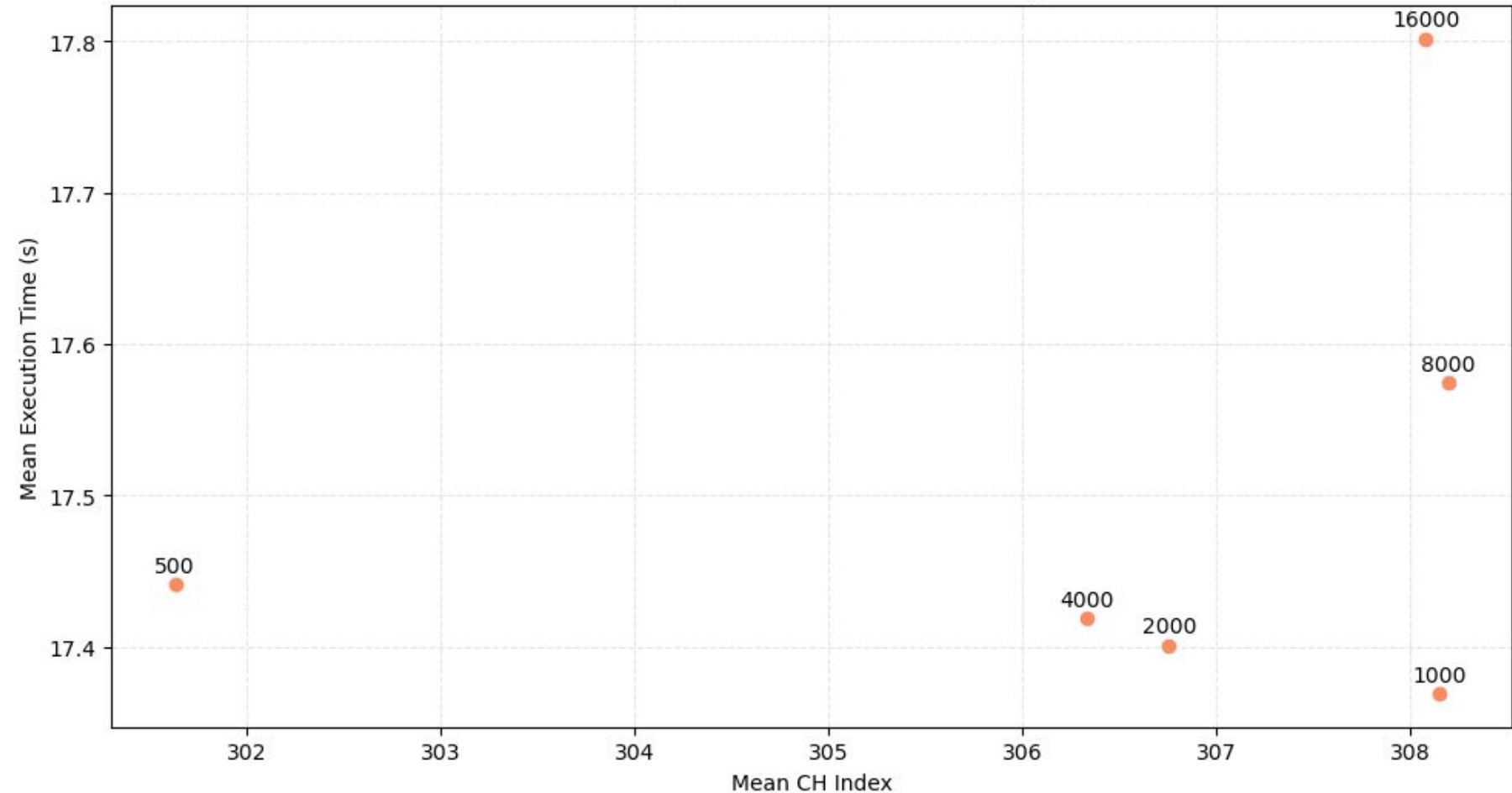



Best Mini Batch Size

Dispersion Plot for Best MiniBatch Size Estimation

Best MiniBatch corresponds to lowest time and highest CH Index

Mini Batch Size
500
1000
2000
4000
8000
16000





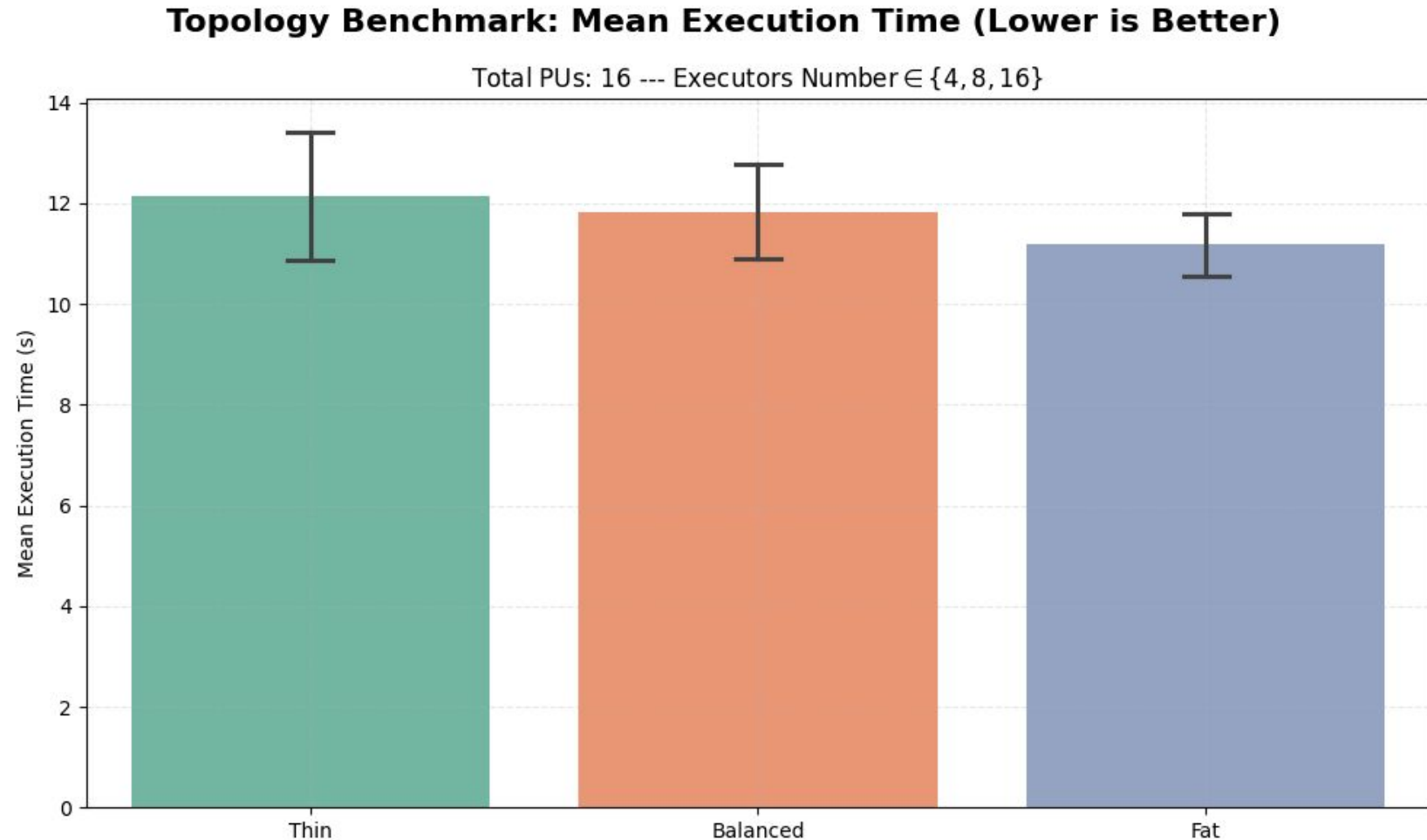
Topology Benchmark

Objective:

Large number of **small executors** or small number of **large executors**.

Which configuration is better?

- **Thin**
1GB + 750MB memory
16 executor x 1 core
- **Balanced**
2GB + 1.5GB memory
8 executor x 2 core
- **Fat**
5GB + 2GB memory
4 executor x 4 core





Topology Benchmark

Cold Start

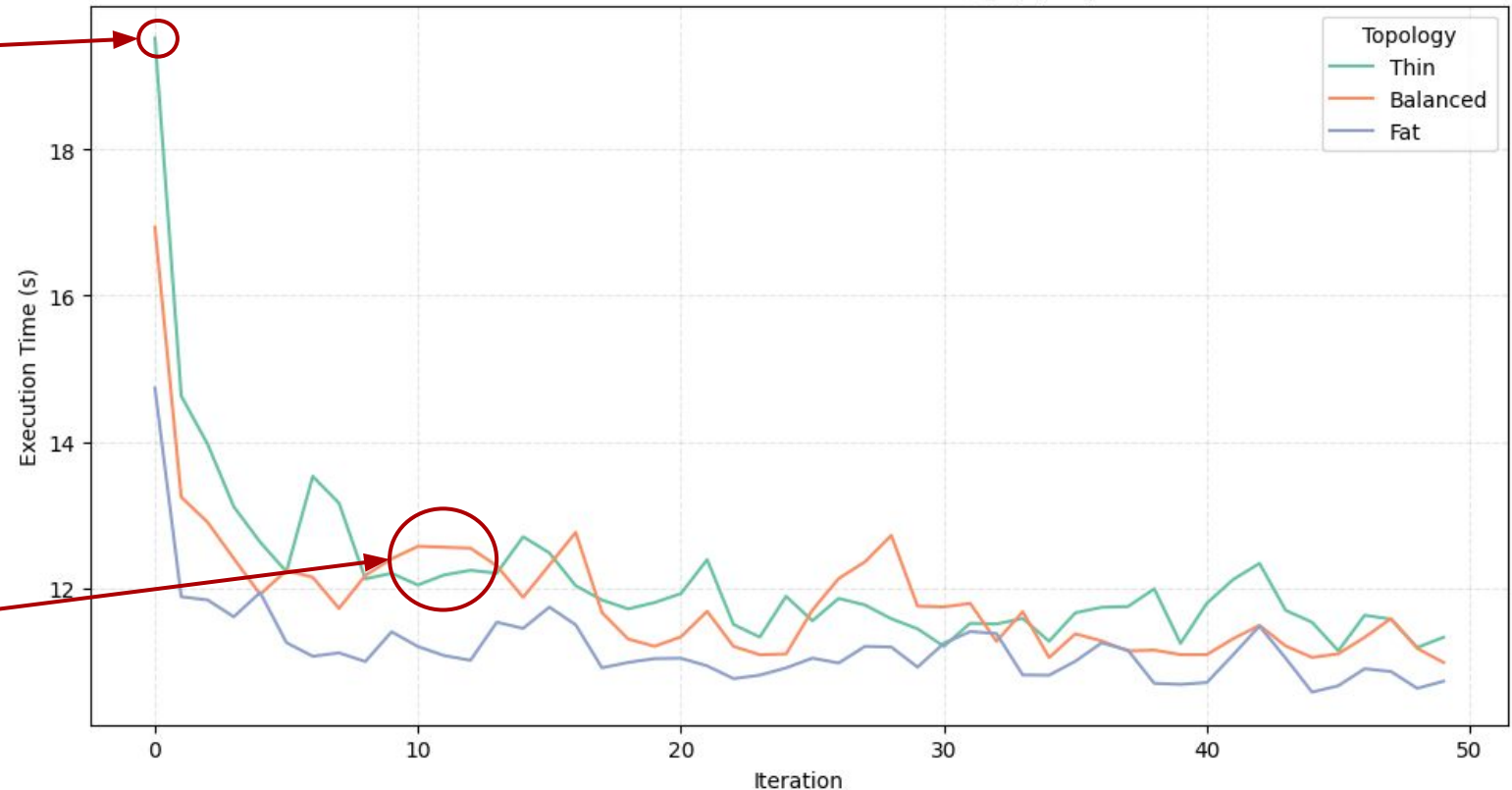
Warm Up

Possible mixing causes

- Straggler Tasks
- Network Congestion

Topology Benchmark: Execution Time over 50 iterations

Total PUs: 16 --- Executors Number $\in \{4, 8, 16\}$





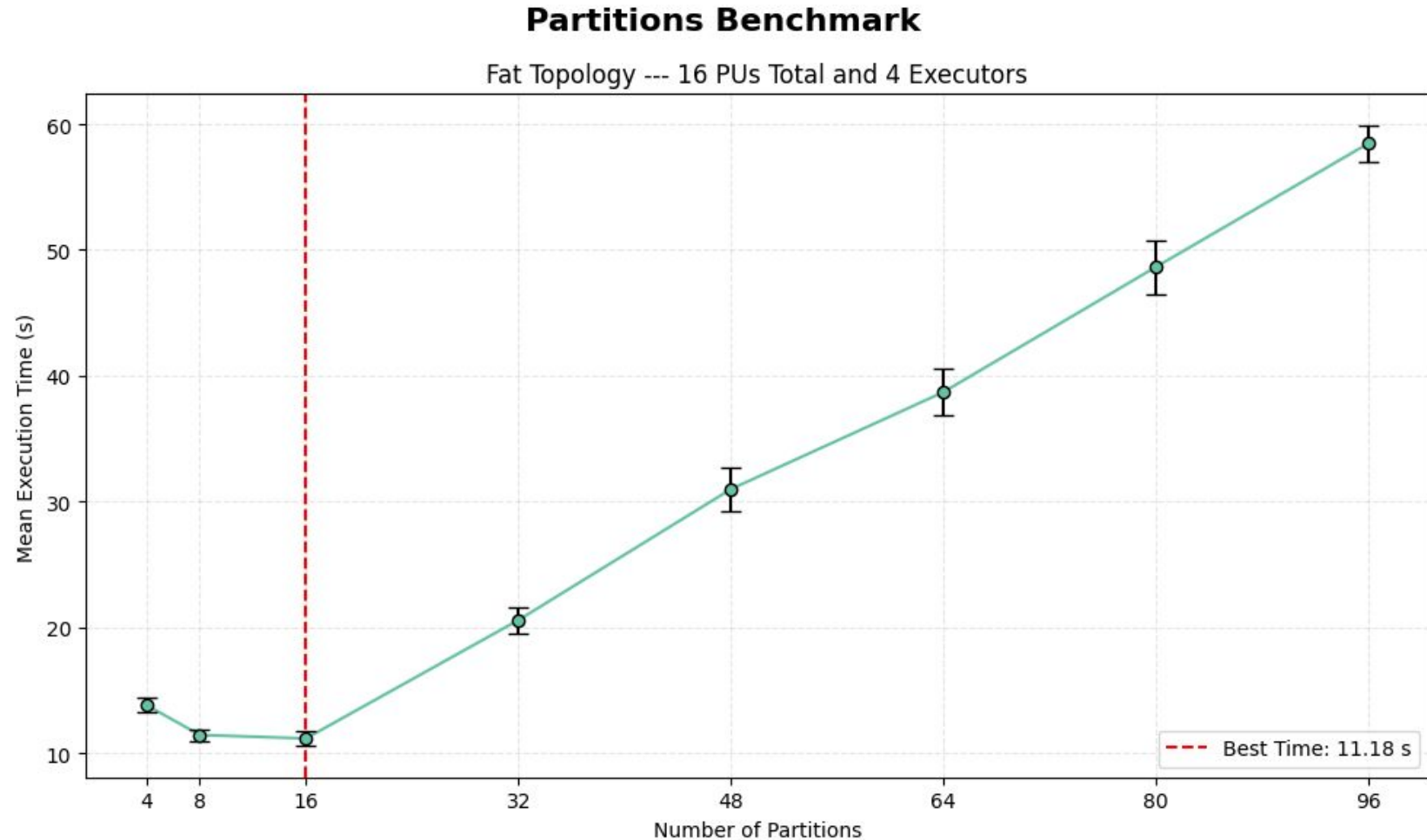
Partitions Benchmark

Objective:

What is the ideal number of partitions to get the best performance?



Expectations met!





Strong Scaling

Objective:

Same problem and topology,
different computation power

Fixed

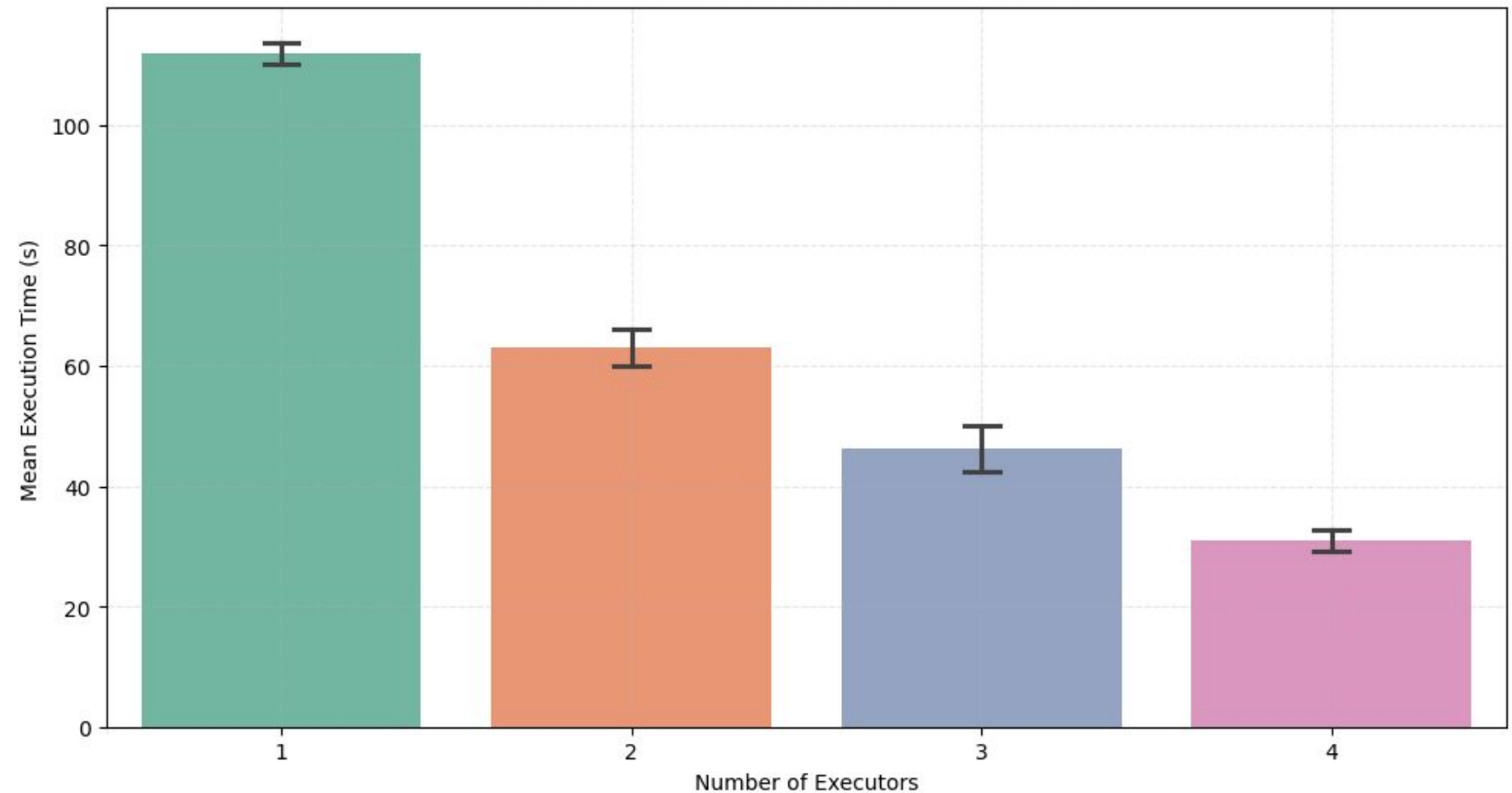
- Partitions: 48
- 4 cores x executor
- 5GB + 2GB memory

Varying

- Total PUs: {4,8,12,16}

Strong Scaling: Mean Execution Time (Lower is Better)

Fixed Executor PUs: 4 --- Fixed Partitions: 48





Strong Scaling

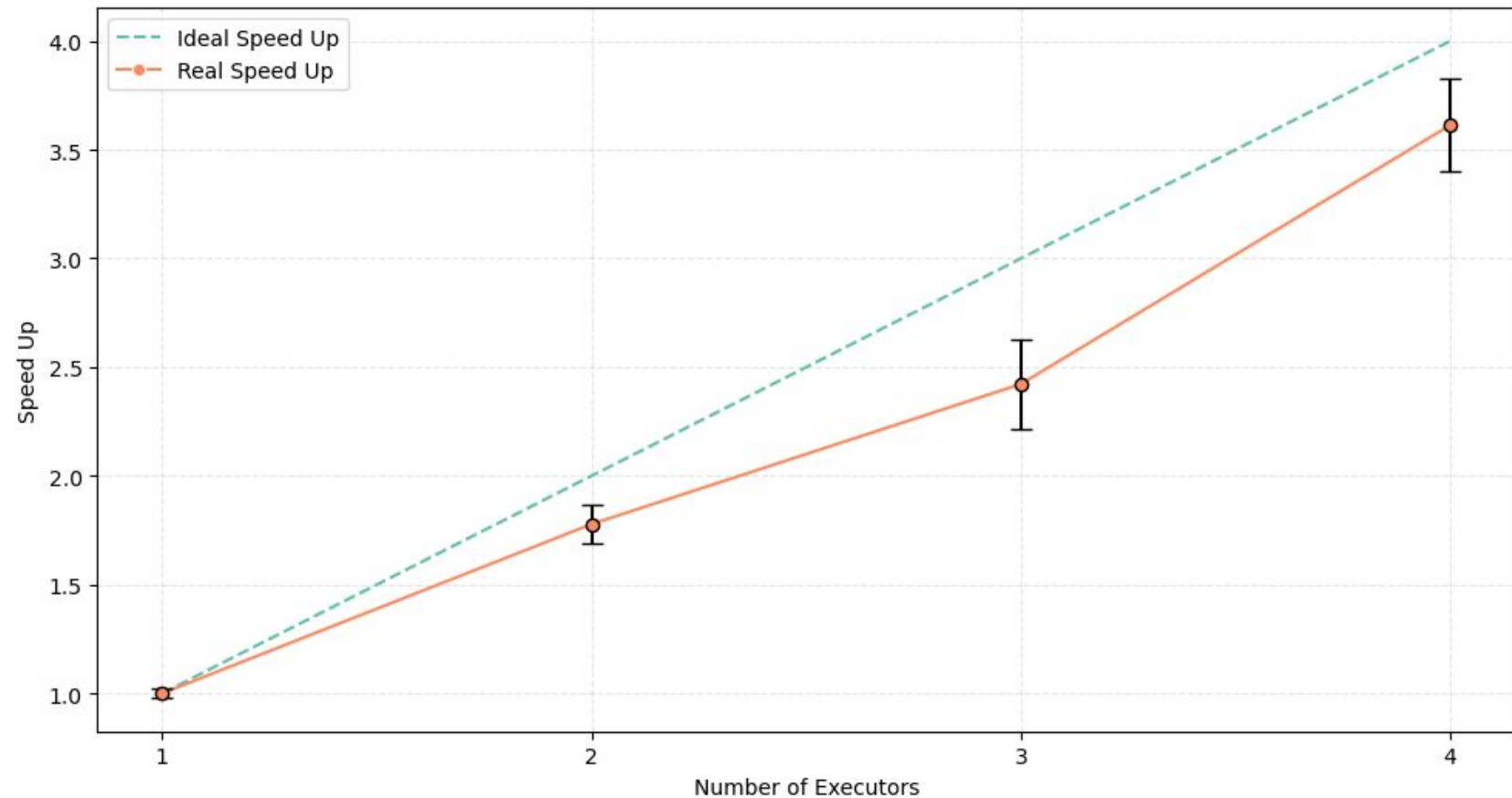
Ideal Speed Up is equivalent
to Amdahl and Gustafson's
Law with $p=1$



**Measured Speed Up takes
into account real computing
factors**

Strong Scaling: Speed Up Analysis

Fixed Executor PUs: 4 --- Fixed Partitions: 48





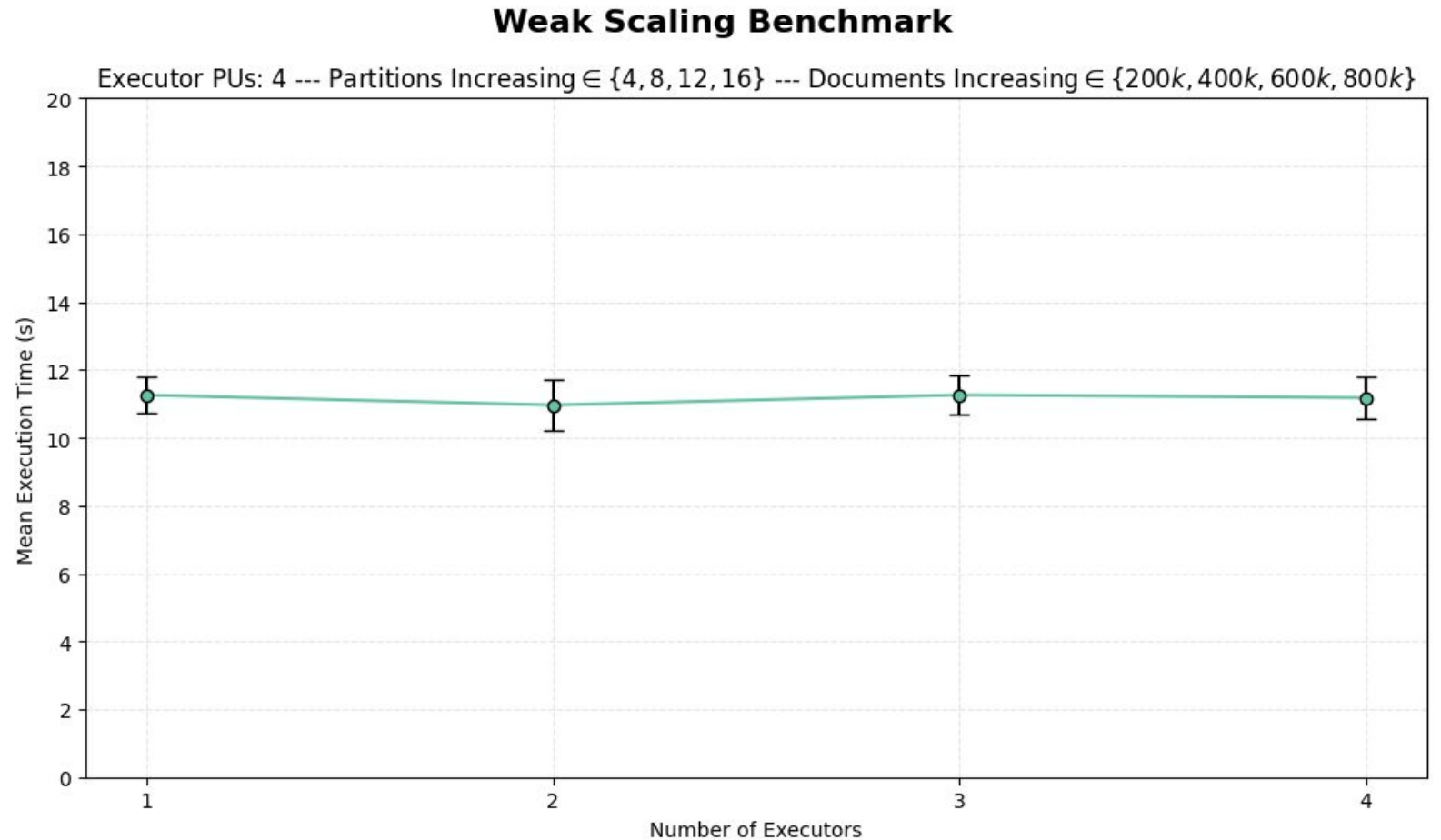
Weak Scaling

Objective:

Evaluate if the system maintains constant performance when **both resources and data size increase proportionally**

Observations:

The execution time remains nearly **flat** across different configurations





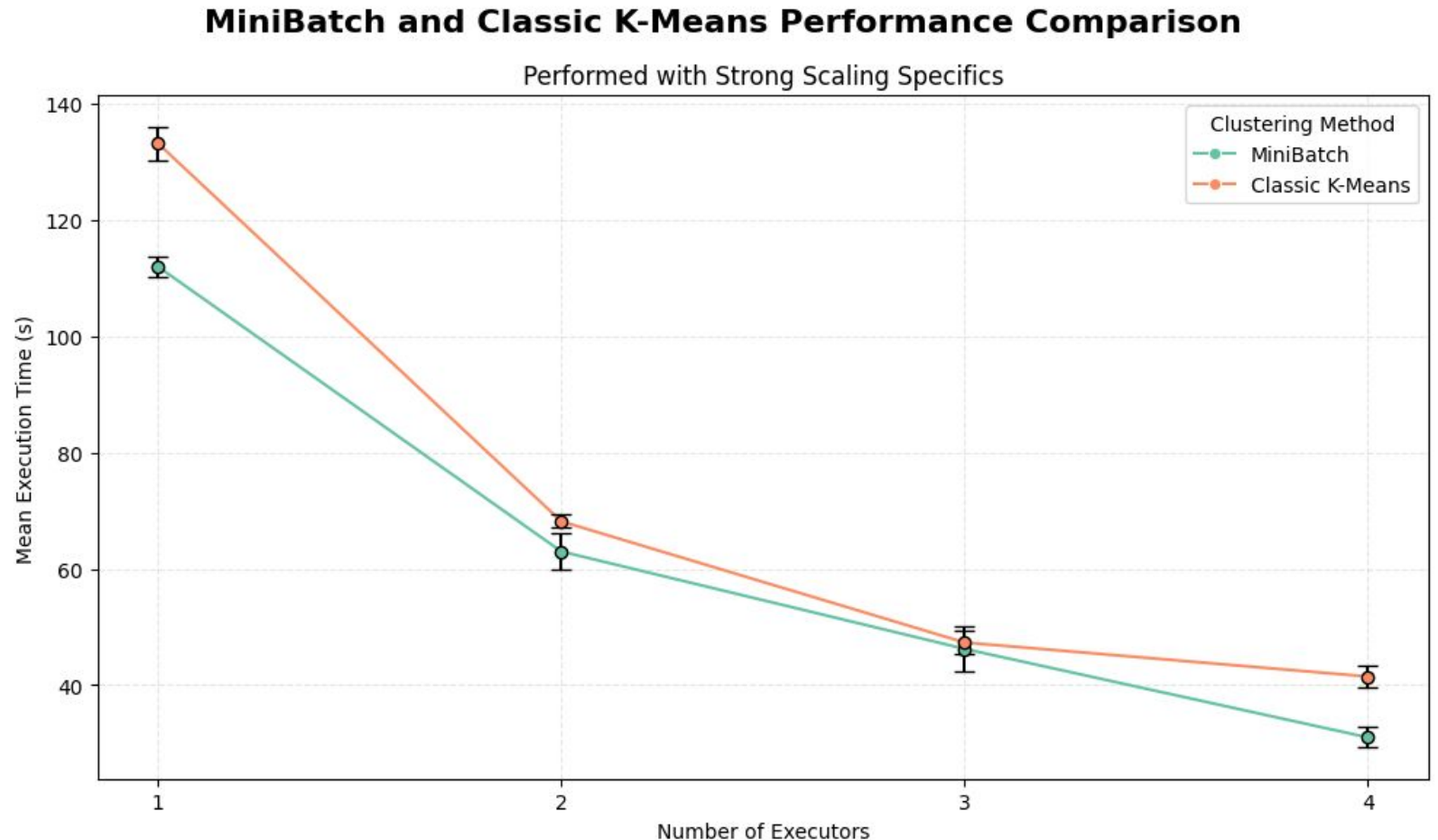
Classic K-Means Comparison

Objective:

Comparison of classic Lloyd's k-Means and Mini-batch implementation

Observations:

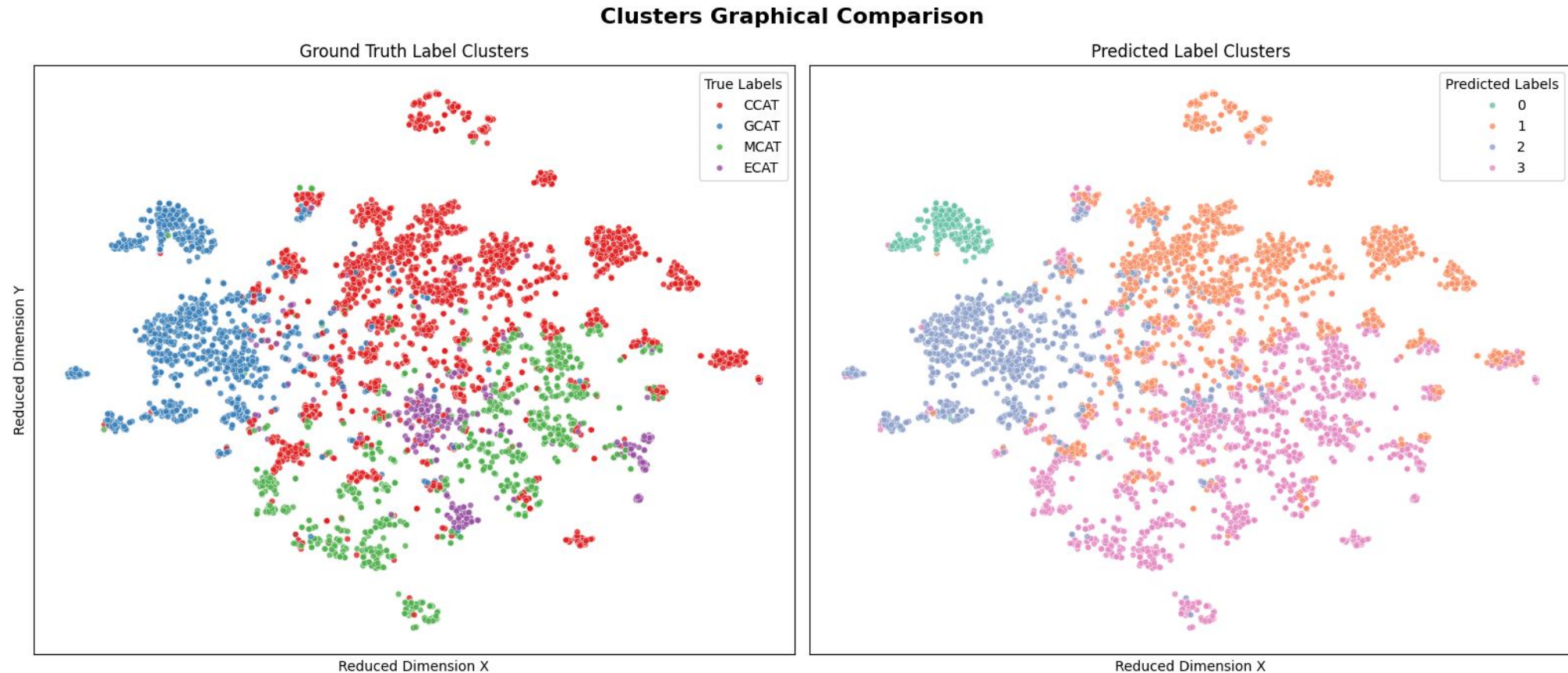
Mini-Batch yields a **slight improvement** in execution time compared to the classic implementation.





Cluster Evaluation

Dimensionality reduction from 100D (SVD) to 2D using t-SNE for visualization





Cluster Evaluation

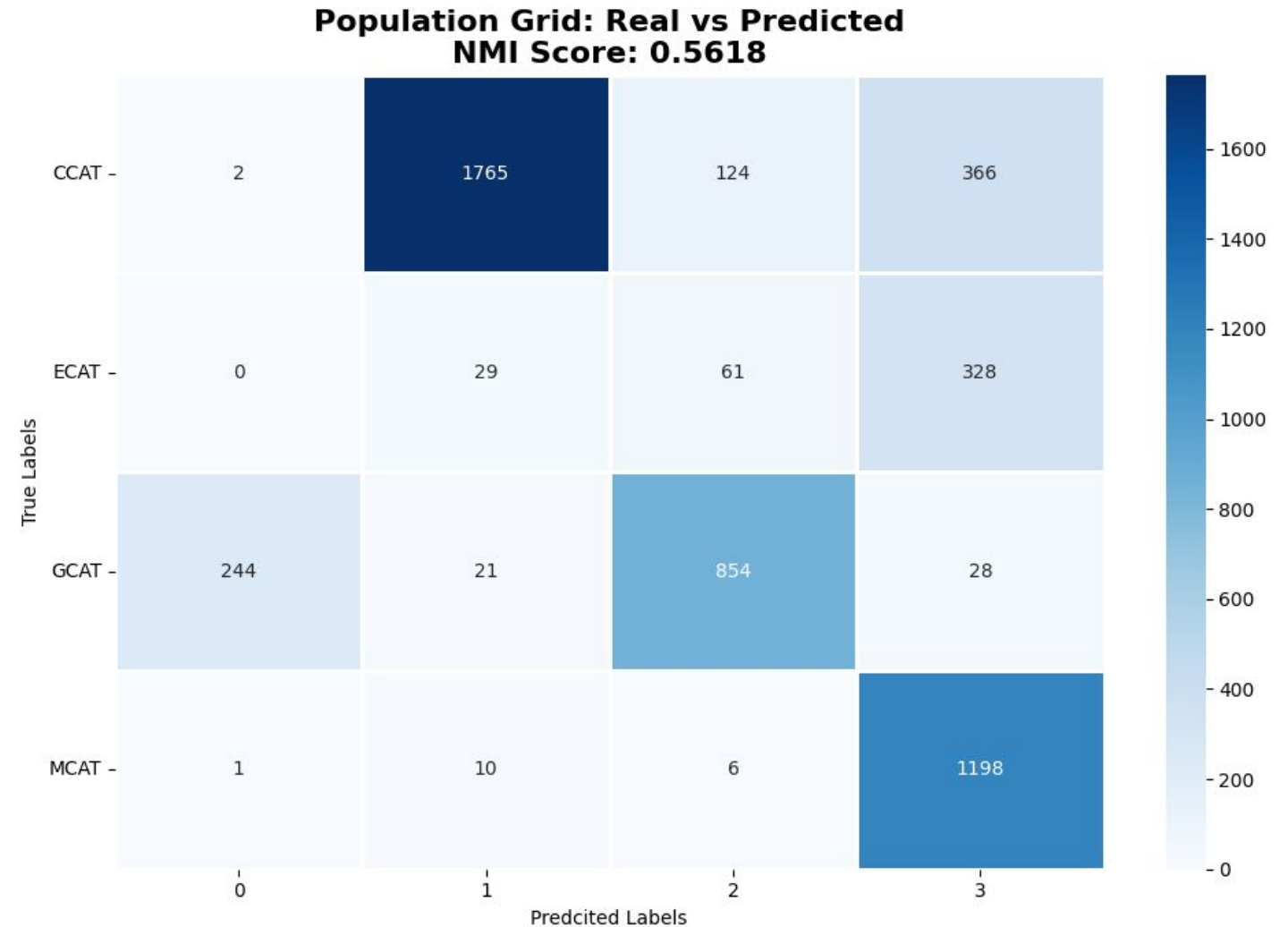
Results:

3 out of 4 clusters approximately represent one of the main topics

The **ECAT** (Economics) category is often misclassified into Cluster 3

NMI Score: 0.5618

(Normalized Mutual Information)



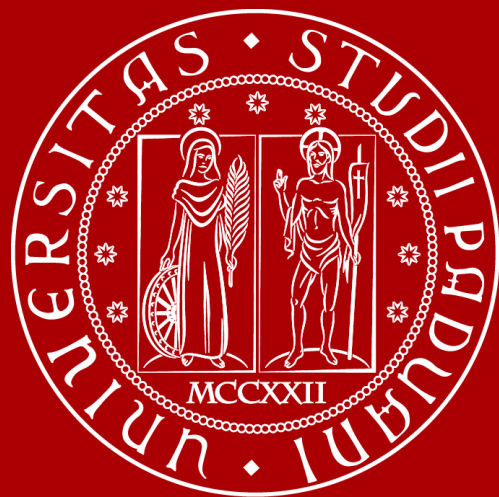


Conclusions

- **Successful Implementation:** Developed Spark-parallelized versions of both D. Sculley's Mini-Batch and Classic K-Means.
- **Benchmarking:** Executed comprehensive scaling tests, achieving performance metrics that align with theoretical expectations.

Fat topology with 16 partition

- **Clustering Validation:** Verified the model's quality by analyzing cluster populations



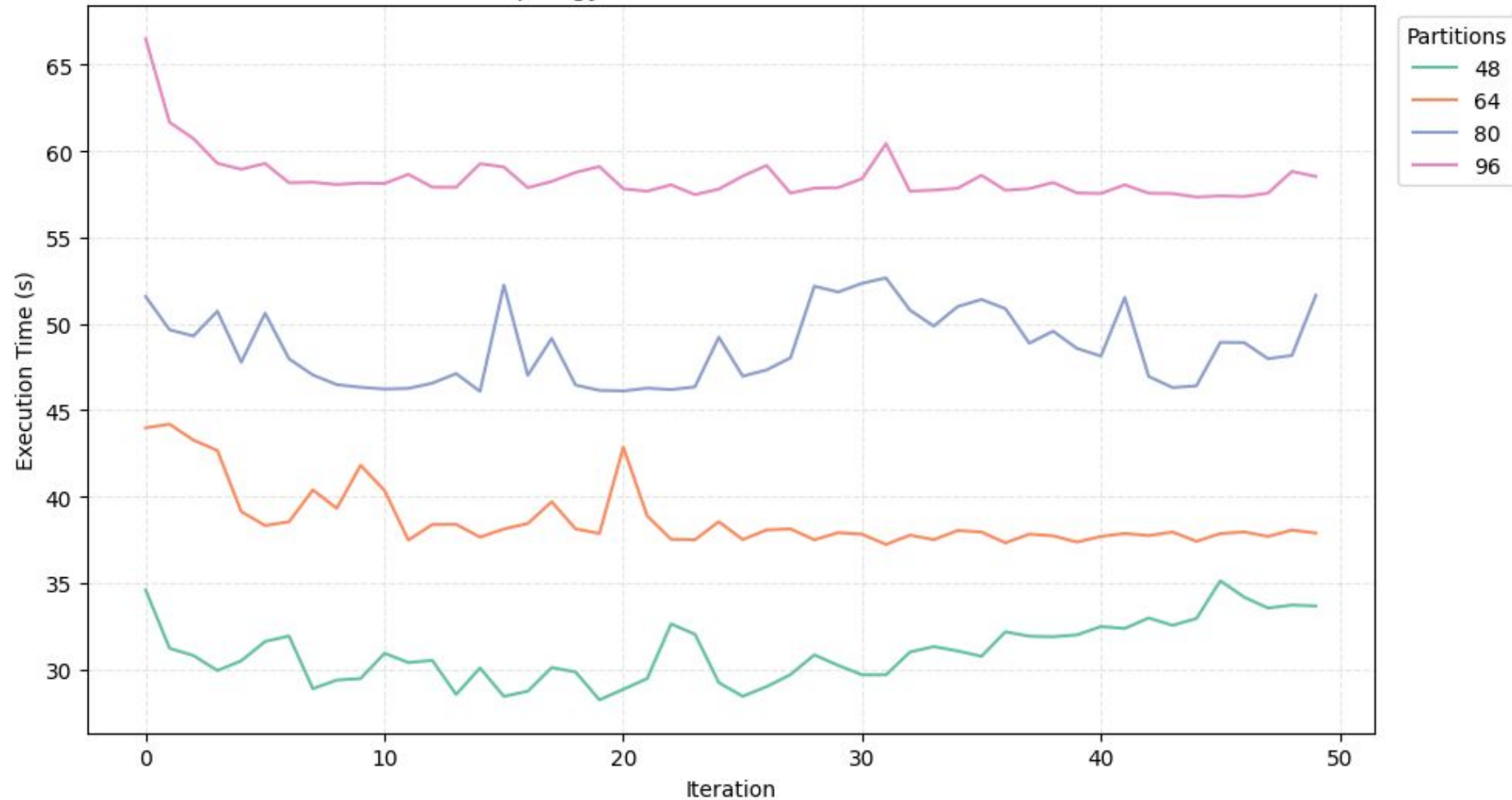
UNIVERSITÀ
DEGLI STUDI
DI PADOVA



Backup Slides

Partitions Benchmark: Fluctuations Analysis

Fat Topology --- 16 PUs Total and 4 Executors





Backup Slides

No horizontal storage scaling: the dataset fits comfortably within the **25 GB** of local storage available on each node.

```
ubuntu@mapdb-2612-master:~/Project$ du -sh data  
1.2G    data
```

Alternatives:

→ Distributed File System

- ◆ Set up a distributed file system across the worker nodes, where each node stores a partition of the dataset
- ◆ This provides a unified view of the data (Spark can access as a single logical storage system)

→ CloudVeneto Object Storage

- ◆ Alternatively, use the shared CloudVeneto Object Storage through the Amazon S3 API
- ◆ We tried it [here](#).